

EECE 5639 Computer Vision - Project 1: Motion Detection Analysis

Matthew Eckert (eckert.mat@northeastern.edu)

February 9, 2026

Abstract

This report details the implementation and analysis of motion detection algorithms using temporal derivatives. The primary objective was to detect moving objects in video sequences by analyzing pixel intensity changes over time. I implemented a pipeline consisting of grayscale conversion, temporal differentiation (using both simple finite differences and Gaussian derivative operators), spatial filtering (box and Gaussian), and adaptive thresholding to generate binary motion masks. The algorithms were tested on three datasets: *Office*, *Red Chair*, and *Crossing*.

Our analysis highlights the critical need to balance spatial and temporal filtering to mitigate both sensor noise and "ghosting" artifacts. I found that the optimal configuration across all datasets was a Gaussian derivative with $\sigma_t = 0.25$ combined with spatial Gaussian smoothing of $\sigma_s = 0.25$. This combination effectively suppressed noise without introducing significant ghosting, which was identified as a major issue in the *Office* and *Red Chair* datasets due to their lower frame capture rates. In the *Crossing* dataset, ghosting only became problematic at higher temporal scales ($\sigma_t \geq 1.0$).

1 Description of Algorithms

The motion detection system processes video sequences frame-by-frame. The core pipeline is described below:

1.1 Preprocessing

Each input video frame $I(x, y, t)$ is first converted from RGB color space to grayscale to simplify intensity analysis. In order to perform this in python, I utilized the **cv2** package's (OpenCV [2]) **cvtColor** function:

```
1 def load_frames_and_grayscale(dir_path):
2
3     frames = []
4     print("[i] Loading frames from
5     directory: ", dir_path)
6
7     files = sorted(os.listdir(dir_path))
8
9     for filename in tqdm.tqdm(files, desc=
10    "Frames to Grayscale"):
11         if filename.endswith(".jpg"):
12             frame = cv2.imread(os.path.
13             join(dir_path, filename))
14             gray = cv2.cvtColor(frame, cv2
15             .COLOR_BGR2GRAY).astype(np.float32)
16             frames.append(gray)
17     return np.array(frames) # Shape: (Time
18     , Height, Width)
```

Listing 1: Python Example

1.2 Temporal Differentiation

To detect motion, I compute the temporal derivative $\frac{\partial I}{\partial t}$ at each pixel. Two methods were implemented:

1.2.1 Simple Difference

A symmetric finite difference operator is applied:

$$\frac{\partial I}{\partial t} \approx \frac{I(t+1) - I(t-1)}{2} \quad (1)$$

This is implemented by convolving the temporal signal with the kernel $[-0.5, 0, 0.5]$. While computa-

tionally efficient, this method is highly sensitive to noise.

```
1 def apply_simple_diff_operator(frames):
2     simple_kernel = np.array([-0.5, 0,
3                               0.5])
4     deriv_simple = scipy.ndimage.
5     convolve1d(frames, simple_kernel, axis
6               =0)
7     # Trim the edges
8     deriv_simple = deriv_simple[1:-1]
9     return deriv_simple
```

Listing 2: Simple Difference Operator

1.2.2 Gaussian Derivative (G'_t)

To improve robustness, I smooth the temporal signal with a Gaussian function $G(t; \sigma_t)$ before differentiation. This is equivalent to convolving with the derivative of a Gaussian kernel $G'(t; \sigma_t)$.

$$\frac{\partial I}{\partial t \text{ smooth}} = I(t) * \frac{\partial G}{\partial t} \quad (2)$$

The smoothing reduces high-frequency noise, making the derivative more indicative of structural motion. I tested temporal scales $\sigma_t \in \{0.25, 0.5, 1.0\}$.

```
1 def apply_gaussian_diff_operator(frames,
2   std_dev):
3     gaussian_res = scipy.ndimage.
4     gaussian_filter1d(frames, sigma=
5     std_dev, axis=0, order=1)
6     # Trim the edges
7     trim_ammount = int(4 * std_dev)
8     gaussian_res = gaussian_res[
9     trim_ammount:-trim_ammount]
10    return gaussian_res
```

Listing 3: Gaussian Derivative Operator

1.3 Spatial Filtering

Before computing the temporal derivative, spatial smoothing is applied to each frame to reduce sensor noise and small texture variations. I compared two spatial filters:

1. **Box Filter:** Computes the local mean within a $k \times k$ window ($k \in \{3, 5\}$).
2. **Gaussian Filter:** Convolves the image with a 2D Gaussian kernel $G(x, y; \sigma_s)$, which provides better preservation of edges compared to the box filter. I tested $\sigma_s \in \{0.25, 0.5, 1.0\}$.

```
1 def apply_2d_spatial_filtering(frames,
2   std_dev, kernel_size, filter_type='
3   gaussian'):
4     if filter_type == 'gaussian':
5         kernel = cv2.getGaussianKernel(
6         kernel_size, std_dev)
7         kernel = kernel @ kernel.T
8     elif filter_type == 'box':
9         kernel = np.ones((kernel_size,
10        kernel_size), np.float32) / (
11        kernel_size * kernel_size)
12
13    filtered_img = cv2.filter2D(src=frames
14    , ddepth=-1, kernel=kernel)
15    return filtered_img
```

Listing 4: Spatial Filtering

1.4 Thresholding

A binary motion mask $M(x, y, t)$ is generated by thresholding the absolute value of the temporal derivative:

$$M(x, y, t) = \begin{cases} 1 & \text{if } \left| \frac{\partial I}{\partial t} \right| > T \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

I used an adaptive threshold $T = k \cdot \sigma_{noise}$, where σ_{noise} is the standard deviation of the temporal derivatives across the frame, assuming the background noise distribution is Gaussian. I set $k = 3.0$ for all experiments.

```
1 def get_adaptive_threshold(derivatives, k
2   =3.0):
3     noise_sigma = np.std(derivatives)
4     return k * noise_sigma
5
6 def form_binary_mask(derivatives,
7   threshold):
8     mask = np.abs(derivatives) > threshold
9     return mask
```

Listing 5: Adaptive Thresholding

1.5 Visualization

Finally, the binary motion mask $M(x, y, t)$ is combined with the original color frame to visualize the detected motion. The mask is overlaid in green on the original image to clearly distinguish moving objects from the background.

2 Experiments and Parameters

The algorithms were evaluated on three datasets exhibiting different motion characteristics:

- **Office:** Indoor scene with a person walking. Medium frame rate.
- **Red Chair:** Chair moved into frame. Low frame rate.
- **Crossing (EnterExitCrossingPaths2cor):** Surveillance with many moving individuals in frame. Standard frame rate.

2.1 Parameter Sweeps

Two primary experiments were conducted to analyze the impact of filter parameters:

2.1.1 Experiment 1: Spatial Box Filter

I fixed the spatial smoothing to a Box Filter of size 3×3 and 5×5 . For each size, I compared the Simple Temporal Difference against Gaussian Temporal Derivatives with $\sigma_t \in \{0.25, 0.5, 1.0\}$.

2.1.2 Experiment 2: Spatial Gaussian Filter

I used a Gaussian spatial filter with varying $\sigma_s \in \{0.25, 0.5, 1.0\}$ (kernel size fixed at 5×5). Similar to Exp. 1, I compared Simple Temporal Difference and Gaussian Temporal Derivatives across the same σ_t range.

3 Observations and Results

3.1 Effect of Temporal Smoothing (σ_t)

The Simple Difference operator ($[-0.5, 0, 0.5]$) proved to be very sensitive to noise, often resulting in "speckled" motion masks even in static regions (false positives). Increasing σ_t in the Gaussian derivative significantly reduced this noise.

However, temporal smoothing introduced a trade-off with *ghosting*.

- **Low σ_t (0.25):** Provided the best balance. It effectively reduced noise while maintaining sharp temporal boundaries.

- **High σ_t (1.0):** Caused significant ghosting. This was particularly severe in the *Office* and *Red Chair* datasets due to their slow frame capture rate; the object moved significantly between frames, and smoothing blended distinct positions, creating wide, inaccurate masks. In the *Crossing* dataset, ghosting was less pronounced at lower σ_t but became an issue at $\sigma_t = 1.0$.

3.2 Effect of Spatial Smoothing

Spatial smoothing was crucial for suppressing texture noise.

- **Box Filter:** Effective at removing noise but tended to blur object boundaries, making the detected shapes blocky.
- **Gaussian Filter:** Provided a better trade-off. Specifically, $\sigma_s = 0.25$ combined with $\sigma_t = 0.25$ yielded the cleanest results across all datasets.

3.3 Effect of Threshold Parameter (k)

The adaptive threshold $T = k \cdot \sigma_{noise}$ plays a crucial role in separating motion from background noise.

- $k = 1$: Selecting a low threshold captures subtle movements but is highly susceptible to sensor noise, resulting in a "noisy" mask with many false positives scattered across the background.
- $k = 3$: Increasing k to 3 raises the bar for detection. This successfully suppresses the background noise, leaving only the significant motion regions corresponding to the moving object.

This trade-off is illustrated in Fig. 4, where $k = 1$ shows granular noise in the background, while $k = 3$ produces a clean segmentation of the walking person.

3.4 Ghosting and Frame Rate Analysis

The "ghosting" artifact, where the moving object appears significantly larger than its actual size or leaves a trail, is exacerbated by two factors: excessive spatial/temporal smoothing and low video frame rates.

In the *Red Chair* dataset, which has a notably low frame rate, the object undergoes large displacements between consecutive frames. When strong spatial smoothing (high σ_s) is applied, the boundaries of the object at time t and $t+1$ are blurred and merged, creating a wide "ghost" region in the difference image.

Figure 1 demonstrates this effect. As the spatial Gaussian σ_s increases, the detected mask expands well beyond the actual chair, losing the object's precise shape. This confirms that for low-frame-rate, high-displacement videos, minimal smoothing is essential to avoid ghosting.

3.5 Representative Results

Below are representative results showing the impact of parameters. In the masks, green indicates detected motion.

4 Conclusions

I successfully implemented a motion detection system based on temporal differentiation. Our experiments verified that balancing spatial and temporal filtering is key to robust detection.

The optimal configuration for all three datasets was found to be a minimal Gaussian smoothing: $\sigma_s = 0.25$ (**Spatial**) and $\sigma_t = 0.25$ (**Temporal**).

I observed that "ghosting" artifacts are heavily influenced by the video frame rate. The *Office* and *Red Chair* datasets, having lower frame rates, were highly susceptible to ghosting even with moderate temporal smoothing. In contrast, the *Crossing* dataset only exhibited ghosting at high smoothing levels ($\sigma_t \geq 1.0$). Therefore, for low-frame-rate surveillance, minimal temporal smoothing is preferred to maintain object localization accuracy.

Future improvements could include background modeling (e.g., Mixture of Gaussians) to handle stationary changes (like lighting) and morphological operations (dilation/erosion) to fill gaps in the detected motion masks.

References

- [1] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
- [2] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [3] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

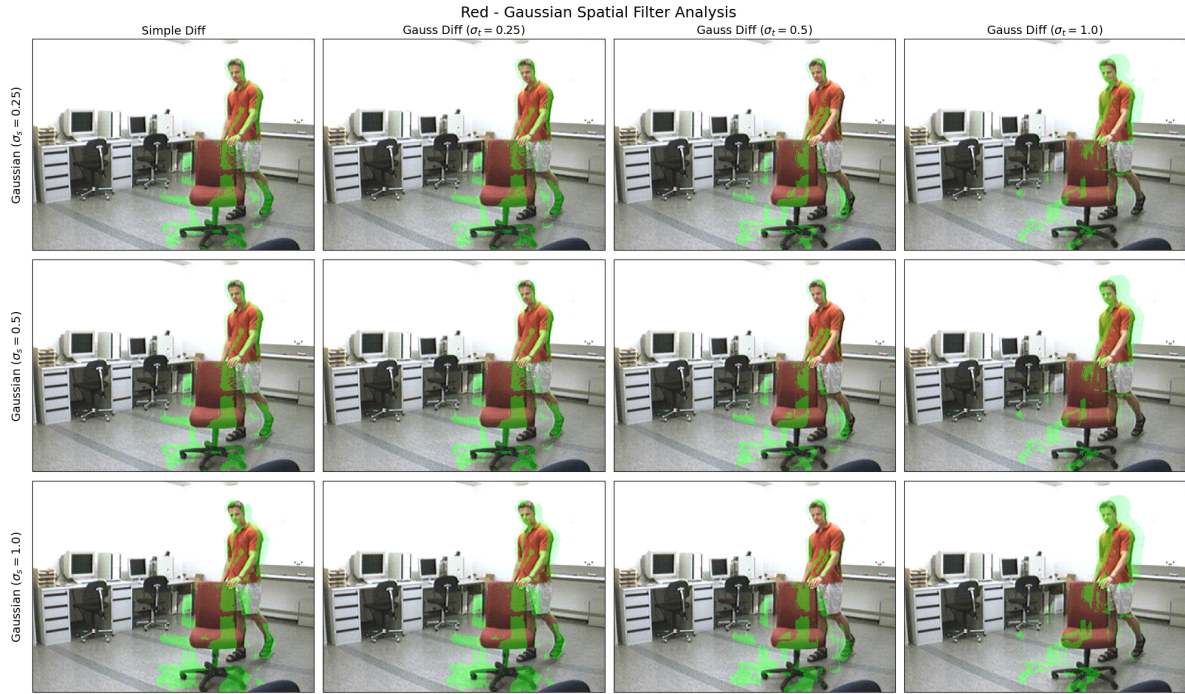


Figure 1: **Ghosting Analysis on Red Chair Dataset.** Comparison of varying spatial Gaussian smoothing (σ_s). Due to the low frame rate, large displacements occur between frames. Increasing σ_s exacerbates the "ghosting" effect, where the mask (green) significantly overestimates the object size.

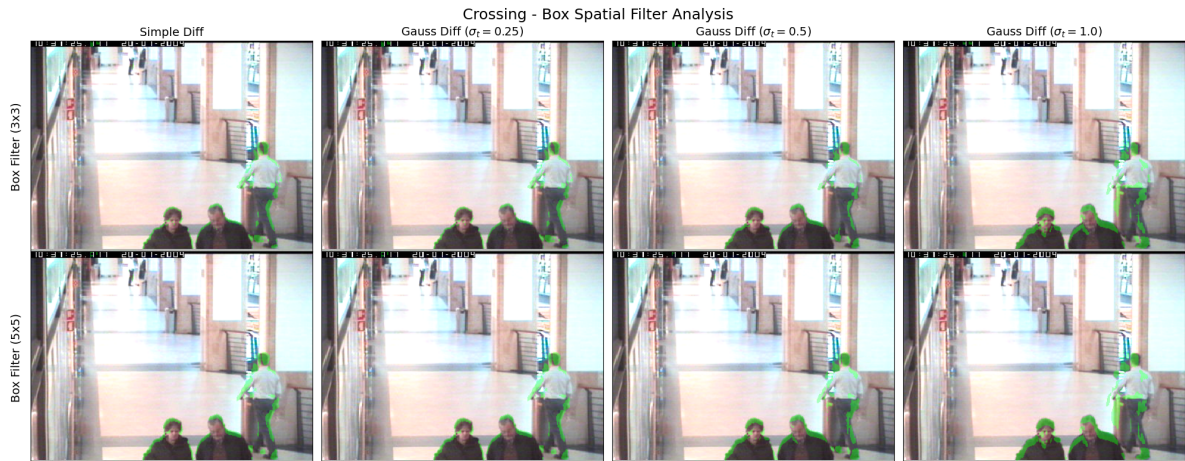


Figure 2: **Crossing Dataset - Box Filter Analysis.** Comparison of 3×3 vs 5×5 box filters with varying temporal derivatives. Note how the 5×5 filter (bottom row) suppresses more noise than the 3×3 filter.



Figure 3: **Office Dataset - Gaussian Spatial Filter.** Analysis of varying spatial σ_s . Using a Gaussian derivative (right columns) yields cleaner masks compared to the simple difference (leftmost column).

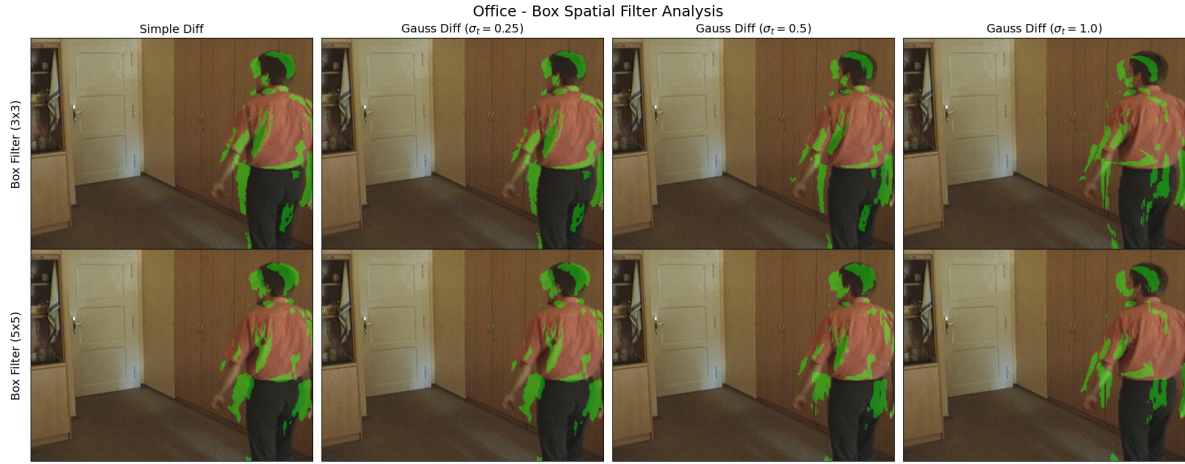


Figure 4: **Impact of Adaptive Threshold Parameter k (Office Dataset).** Using a simple difference operator with a 3×3 box filter. (a) At $k = 1$, the threshold is too low, capturing significant sensor noise in the background. (b) At $k = 3$, the noise is suppressed, isolating the moving person.